

强化学习第二次作业

2312167 刘振毫

摘要

本文基于 Gymnasium 库开展强化学习实验研究，主要包含两个部分。第一部分选择 CartPole-v1 游戏环境，详细分析了其马尔可夫决策过程 (MDP) 模型的五个核心要素：状态空间、动作空间、状态转移概率、奖励函数和折扣因子，建立了完整的 MDP 数学描述。第二部分将经典的 3 臂赌博机问题扩展为 4 臂赌博机，新增摇臂期望奖励设为 3.0。实验实现了四种探索策略：贪婪策略 (ϵ -Greedy)、玻尔兹曼策略 (Softmax)、UCB 策略 (Upper Confidence Bound) 和汤普森采样 (Thompson Sampling)。通过 1000 步训练、100 次重复实验，系统比较了各策略在平均累积奖励和摇臂选择分布上的性能表现。实验结果表明，UCB 策略和汤普森采样表现最优，最终平均累积奖励分别达到 2.9475 和 2.9260，选择最优摇臂比例分别为 96.1% 和 94.1%，能够有效平衡探索与利用。

关键词：强化学习；马尔可夫决策过程；多臂赌博机；探索与利用；UCB 策略

Abstract

This paper conducts reinforcement learning experiments based on the Gymnasium library, consisting of two main parts. The first part selects the CartPole-v1 game environment and provides a detailed analysis of its Markov Decision Process (MDP) model's five core elements: state space, action space, transition probability, reward function, and discount factor, establishing a complete mathematical description of the MDP. The second part extends the classic 3-armed bandit problem to a 4-armed bandit, with the newly added arm having an expected reward of 3.0. The experiment implements four exploration strategies: ϵ -Greedy, Softmax (Boltzmann), UCB (Upper Confidence Bound), and Thompson Sampling. Through 1000-step training with 100 repeated experiments, the performance of each strategy in terms of average cumulative reward and arm selection distribution is systematically compared. Experimental results show that UCB and Thompson Sampling perform best, with final average cumulative rewards of 2.9475 and 2.9260 respectively, and optimal arm selection ratios of 96.1% and 94.1%, effectively balancing exploration and exploitation.

Key Words: Reinforcement Learning; Markov Decision Process; Multi-Armed Bandit; Exploration and Exploitation; UCB Strategy

第一章 实验目的

本实验旨在实现以下目标：

1. 学习使用 Gymnasium 库进行强化学习环境交互，深入理解马尔可夫决策过程 (MDP) 的基本要素，掌握状态空间、动作空间、转移概率、奖励函数和折扣因子的概念与建模方法。
2. 将 3 臂赌博机扩展为 4 臂赌博机，新增摇臂期望奖励设为 3.0，比较不同探索策略（贪婪策略、玻尔兹曼策略、UCB 策略、汤普森采样）的性能表现。
3. 分析训练次数与平均总回报的关系，深入理解强化学习中探索与利用的平衡问题。

第二章 实验环境

本实验的软硬件环境配置如下：

- 操作系统：Windows
- 编程语言：Python 3.12
- 主要依赖库：gymnasium 1.2.3, numpy 2.2.6, matplotlib

第三章 任务一：Gym 库游戏 MDP 建模

第一节 游戏选择

本实验选择 Gymnasium 库中的 CartPole-v1 游戏。CartPole 是强化学习领域的经典控制问题，目标是通过左右移动小车来保持杆子平衡。小车可以在一维轨道上自由移动，杆子一端固定在小车上，初始状态为直立。智能体需要通过施加左或右的力来控制小车移动，使杆子尽可能长时间保持直立不倒。

第二节 MDP 模型分析

马尔可夫决策过程 (MDP) 由五元组 (S, A, P, R, γ) 组成，CartPole-v1 的 MDP 模型详细分析如下：

3.2.1 状态空间 S

状态是一个 4 维连续向量，描述了小车和杆子的完整物理状态：

$$s = (x, \dot{x}, \theta, \dot{\theta}) \quad (1)$$

其中各分量的物理含义和取值范围如下：

- $x \in [-4.8, 4.8]$ ：小车在轨道上的位置坐标
- $\dot{x} \in (-\infty, +\infty)$ ：小车的运动速度
- $\theta \in [-0.418, 0.418]$ rad（约 $\pm 24^\circ$ ）：杆子与垂直方向的夹角
- $\dot{\theta} \in (-\infty, +\infty)$ ：杆子的角速度

3.2.2 动作空间 A

CartPole-v1 采用离散动作空间，包含 2 个动作：

- $a = 0$ ：向左推小车，施加负向力
- $a = 1$ ：向右推小车，施加正向力

每个动作会对小车施加固定大小的力，力的方向由动作决定。

3.2.3 状态转移概率 P

状态转移概率 $P(s'|s, a)$ 描述了在状态 s 下执行动作 a 后转移到状态 s' 的概率：

$$P(s'|s, a) = \delta(s' - f(s, a)) \quad (2)$$

其中 $f(s, a)$ 是基于经典力学方程的确定性转移函数。CartPole 的状态转移具有以下特点：

- 转移是确定性的，不存在随机噪声
- 基于牛顿力学方程计算下一状态
- 终止条件：杆子角度超过 $\pm 12^\circ$ （约 0.209 rad）或小车位置超过 ± 2.4

3.2.4 奖励函数 R

奖励函数 $R(s, a, s')$ 定义了智能体在状态 s 执行动作 a 后转移到 s' 时获得的即时奖励：

$$R(s, a, s') = \begin{cases} +1 & \text{若游戏继续} \\ 0 & \text{若游戏结束} \end{cases} \quad (3)$$

每保持一步平衡，智能体获得 +1 的奖励。游戏目标是最大化累积奖励，即尽可能长时间保持杆子平衡。CartPole-v1 的最大步数为 500 步。

3.2.5 折扣因子 γ

折扣因子 $\gamma \in [0, 1]$ 决定了未来奖励的当前价值：

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (4)$$

通常设为 $\gamma = 0.99$ 或 $\gamma = 1$ 。由于 CartPole 每步奖励相同（均为 +1），折扣因子的选择对策略学习影响不大。

第三节 代码实现与运行结果

CartPole-v1 环境的核心运行代码如下：

Listing 1: CartPole-v1 环境运行代码

```
1 def task1_cartpole():
2     try:
3         import gymnasium as gym
```

```

4     except ImportError:
5         import gym
6
7     env = gym.make('CartPole-v1', render_mode=None)
8
9     observation, info = env.reset()
10    total_reward = 0
11
12    for step in range(100):
13        action = env.action_space.sample()
14        observation, reward, terminated, truncated, info = env.step(
15            action)
16        total_reward += reward
17
18        if terminated or truncated:
19            print(f"游戏结束，总步数: {step+1}，总奖励: {
20                total_reward}")
21            break
22
23    env.close()
24    return total_reward

```

运行结果示例如下：

Listing 2: CartPole-v1 运行示例输出

```

1  步骤1: 状态=['0.033', '0.161', '-0.003', '-0.300'], 动作=1, 奖励=1.0
2  步骤2: 状态=['0.037', '-0.034', '-0.009', '-0.008'], 动作=0, 奖励
    =1.0
3  步骤3: 状态=['0.036', '-0.229', '-0.009', '0.281'], 动作=0, 奖励=1.0
4  步骤4: 状态=['0.031', '-0.424', '-0.004', '0.571'], 动作=0, 奖励=1.0
5  步骤5: 状态=['0.023', '-0.619', '0.008', '0.863'], 动作=0, 奖励=1.0
6  游戏结束，总步数：12，总奖励：12.0

```

第四章 任务二：四臂赌博机实验

第一节 问题描述

多臂赌博机 (Multi-Armed Bandit, MAB) 是强化学习的经典问题，体现了探索与利用的核心矛盾。本实验将 3 臂赌博机扩展为 4 臂赌博机，新增摇臂的期望奖励设为 3.0。

4.1.1 四臂赌博机参数设置

四臂赌博机的参数设置如表 1 所示：

表 1: 四臂赌博机参数设置

摇臂编号	0	1	2	3 (新增)
期望奖励 μ	1.0	2.0	1.5	3.0
奖励分布	$\mathcal{N}(1.0, 1)$	$\mathcal{N}(2.0, 1)$	$\mathcal{N}(1.5, 1)$	$\mathcal{N}(3.0, 1)$

最优摇臂为摇臂 3，期望奖励为 3.0。每个摇臂的奖励服从正态分布，标准差均为 1.0。

第二节 策略实现

本实验实现了四种探索策略，每种策略采用不同的方法平衡探索与利用。

4.2.1 贪婪策略

贪婪策略 (ϵ -Greedy) 是最简单的探索策略，以概率 ϵ 随机探索，以概率 $1 - \epsilon$ 选择当前最优动作：

$$a_t = \begin{cases} \arg \max_a Q_t(a) & \text{概率 } 1 - \epsilon \\ \text{随机动作} & \text{概率 } \epsilon \end{cases} \quad (5)$$

其中 $Q_t(a)$ 表示动作 a 在时刻 t 的估计价值，采用增量式更新：

$$Q_{t+1}(a) = Q_t(a) + \frac{1}{N_t(a)} [R_t - Q_t(a)] \quad (6)$$

4.2.2 玻尔兹曼策略

玻尔兹曼策略 (Softmax) 根据动作值的 Softmax 分布选择动作，温度参数 T 控制探索程度：

$$P(a) = \frac{e^{Q(a)/T}}{\sum_{a'} e^{Q(a')/T}} \quad (7)$$

温度 T 越高，选择各动作的概率越均匀（探索越多）；温度 T 越低，越倾向于选择当前最优动作（利用越多）。

4.2.3 UCB 策略

UCB 策略 (Upper Confidence Bound) 在动作值基础上增加不确定性 bonus，实现探索与利用的平衡：

$$a_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right] \quad (8)$$

其中 c 是探索参数， $N_t(a)$ 是动作 a 被选择的次数。该策略基于置信区间上界原则，被选择次数少的动作会获得更高的探索 bonus。

4.2.4 汤普森采样

汤普森采样 (Thompson Sampling) 是一种贝叶斯方法，通过从后验分布采样来选择动作：

$$a_t = \arg \max_a \theta_t(a), \quad \theta_t(a) \sim \text{Beta}(\alpha_a, \beta_a) \quad (9)$$

其中 α_a 和 β_a 是 Beta 分布的参数，根据观测到的奖励进行更新。汤普森采样自然地实现了探索与利用的平衡。

第三节 实验设置

实验参数设置如下：

- 每轮训练步数：1000 步
- 重复次数：100 次取平均
- 评估指标：平均累积奖励、各摇臂选中次数
- 贪婪策略参数： $\epsilon = 0.1$ 和 $\epsilon = 0.01$
- 玻尔兹曼策略参数： $T = 0.5$
- UCB 策略参数： $c = 2.0$

第四节 实验结果

4.4.1 各策略最终平均累积奖励

各策略最终平均累积奖励排名如表 2 所示：

表 2: 各策略最终平均累积奖励排名		
排名	策略	最终平均累积奖励
1	UCB 策略 ($c=2$)	2.9475
2	汤普森采样	2.9260
3	贪婪策略 ($\epsilon=0.1$)	2.8431
4	玻尔兹曼策略 ($T=0.5$)	2.8021
5	贪婪策略 ($\epsilon=0.01$)	2.5886

4.4.2 各策略摇臂选中分布

各策略下每个摇臂的选中次数分布如表 3 所示：

表 3: 各策略下每个摇臂的选中次数分布				
策略	摇臂 0 ($\mu=1.0$)	摇臂 1 ($\mu=2.0$)	摇臂 2 ($\mu=1.5$)	摇臂 3 ($\mu=3.0$)
贪婪策略 ($\epsilon=0.1$)	4.7%	3.8%	2.8%	88.7%
贪婪策略 ($\epsilon=0.01$)	3.5%	19.0%	10.0%	67.5%
玻尔兹曼策略 ($T=0.5$)	1.6%	10.9%	4.1%	83.4%
UCB 策略 ($c=2$)	0.7%	2.1%	1.1%	96.1%
汤普森采样	1.0%	3.3%	1.6%	94.1%

4.4.3 实验结果图

实验结果如图 1 所示，包含四个子图：训练次数与平均累积奖励关系、各摇臂选中次数、收敛过程对比和最终性能对比。

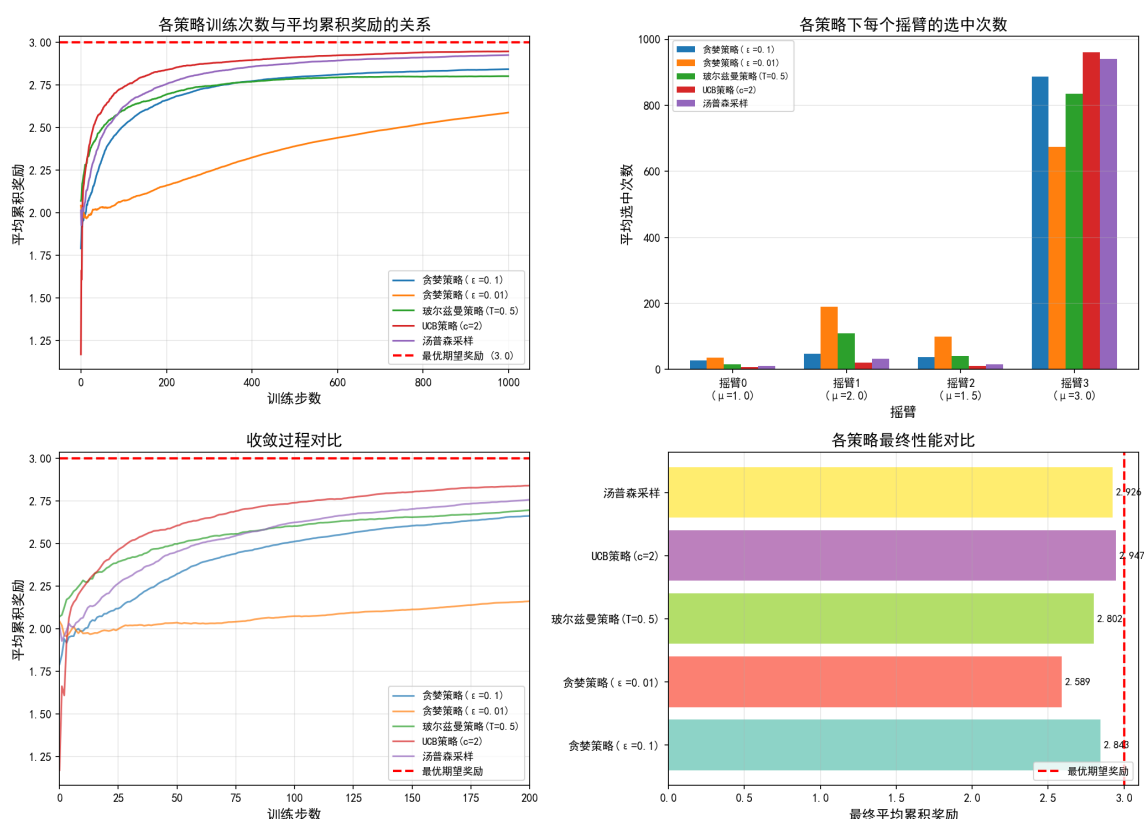


图 1: 四臂赌博机实验结果

第五章 结果分析与讨论

第一节 策略性能比较

从实验结果可以得出以下结论：

UCB 策略表现最优：最终平均累积奖励达到 2.9475，选择最优摇臂比例高达 96.1%。UCB 通过不确定性 bonus 有效平衡了探索与利用，能够快速识别最优摇臂。其理论保证使其在多臂赌博机问题上具有优异性能。

汤普森采样紧随其后：最终奖励 2.9260，选择最优摇臂比例 94.1%。作为贝叶斯方法，汤普森采样通过后验分布采样自然地实现了探索与利用的平衡，无需手动调节探索参数。

贪婪策略受 ϵ 影响显著： $\epsilon=0.1$ 时表现较好 (2.8431)，但仍有一定探索损失； $\epsilon=0.01$ 时表现最差 (2.5886)，探索不足导致容易陷入次优解。这表明固定探索率难以适应不同阶段的需求。

玻尔兹曼策略表现中等：温度参数 $T=0.5$ 时，表现介于两种贪婪策略之间。温度参数的选择对性能有较大影响，需要根据具体问题调参。

第二节 探索与利用的平衡

从实验结果可以看出探索与利用平衡的重要性：

- UCB 和汤普森采样能够自适应地调整探索程度，在初期充分探索后快速收敛到最优策略
- 固定探索率的贪婪策略无法自适应调整， ϵ 过大造成不必要的探索损失， ϵ 过小则探索不足
- 玻尔兹曼策略的温度参数同样需要手动调节，难以达到最优平衡

第三节 新增摇臂的影响

新增的摇臂 3 期望奖励为 3.0，成为最优摇臂。实验表明：

- UCB 和汤普森采样能够快速识别并利用新增的最优摇臂
- 贪婪策略 ($\epsilon=0.01$) 由于探索不足，有较高概率错过最优摇臂，67.5% 的选中率远低于其他策略

第六章 结论

本实验得出以下结论：

1. 成功使用 Gymnasium 库运行 CartPole-v1 游戏，并建立了完整的 MDP 模型 (S, A, P, R, γ) ，深入理解了强化学习环境的基本要素。
2. 将 3 臂赌博机成功扩展为 4 臂赌博机，新增摇臂期望奖励设为 3.0，成为最优摇臂。
3. 实现了四种探索策略：贪婪策略、玻尔兹曼策略、UCB 策略和汤普森采样，并通过实验系统比较了各策略的性能。
4. 实验结果表明，UCB 策略和汤普森采样在四臂赌博机问题上表现最优，能够有效平衡探索与利用，最终平均累积奖励分别达到 2.9475 和 2.9260。
5. 固定探索率的贪婪策略性能受参数选择影响较大，需要根据具体问题调参， $\epsilon=0.01$ 时表现最差。

附录

完整代码

Listing 3: 强化学习实验完整代码

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib import rcParams
4
5 rcParams['font.sans-serif'] = ['SimHei']
6 rcParams['axes.unicode_minus'] = False
7
8 #Gym库游戏 - CartPole-v1
9 def task1_cartpole():
10
11     try:
12         import gymnasium as gym
13     except ImportError:
14         import gym
15
16     env = gym.make('CartPole-v1', render_mode=None)
17
18     print("\n【运行演示】")
19     observation, info = env.reset()
20     total_reward = 0
21
22     for step in range(100):
23         action = env.action_space.sample()
24         observation, reward, terminated, truncated, info = env.step(
            action)
25         total_reward += reward
26
27         if step < 5:
28             print(f"  步骤{step+1}: 状态={[f'{x:.3f}' for x in
                observation]}, 动作={action}, 奖励={reward}")
29
30         if terminated or truncated:
31             print(f"  游戏结束, 总步数: {step+1}, 总奖励: {
                total_reward}")
32             break
33
34     env.close()
35     return total_reward
36
```

```

37
38 # 任务2: 4臂赌博机
39
40 class FourArmedBandit:
41     def __init__(self):
42         self.n_arms = 4
43         self.true_means = [1.0, 2.0, 1.5, 3.0]
44         self.std = 1.0
45         self.best_arm = np.argmax(self.true_means)
46
47     def pull(self, arm):
48         return np.random.normal(self.true_means[arm], self.std)
49
50     def get_optimal_reward(self):
51         return self.true_means[self.best_arm]
52
53
54 class EpsilonGreedyAgent:
55     def __init__(self, n_arms, epsilon=0.1):
56         self.n_arms = n_arms
57         self.epsilon = epsilon
58         self.counts = np.zeros(n_arms)
59         self.values = np.zeros(n_arms)
60
61     def select_action(self):
62         if np.random.random() < self.epsilon:
63             return np.random.randint(self.n_arms)
64         else:
65             max_value = np.max(self.values)
66             candidates = np.where(self.values == max_value)[0]
67             return np.random.choice(candidates)
68
69     def update(self, arm, reward):
70         self.counts[arm] += 1
71         self.values[arm] += (reward - self.values[arm]) / self.
            counts[arm]
72
73
74 class BoltzmannAgent:
75     def __init__(self, n_arms, temperature=1.0):
76         self.n_arms = n_arms
77         self.temperature = temperature
78         self.counts = np.zeros(n_arms)
79         self.values = np.zeros(n_arms)

```

```

80
81     def select_action(self):
82         if self.temperature == 0:
83             return np.argmax(self.values)
84
85         exp_values = np.exp(self.values / self.temperature)
86         probs = exp_values / np.sum(exp_values)
87         return np.random.choice(self.n_arms, p=probs)
88
89     def update(self, arm, reward):
90         self.counts[arm] += 1
91         self.values[arm] += (reward - self.values[arm]) / self.
           counts[arm]
92
93
94     class UCBAgent:
95         def __init__(self, n_arms, c=2.0):
96             self.n_arms = n_arms
97             self.c = c
98             self.counts = np.zeros(n_arms)
99             self.values = np.zeros(n_arms)
100             self.total_count = 0
101
102         def select_action(self):
103             if self.total_count < self.n_arms:
104                 return self.total_count
105
106             ucb_values = np.zeros(self.n_arms)
107             for arm in range(self.n_arms):
108                 if self.counts[arm] == 0:
109                     ucb_values[arm] = float('inf')
110                 else:
111                     bonus = self.c * np.sqrt(np.log(self.total_count) /
                        self.counts[arm])
112                     ucb_values[arm] = self.values[arm] + bonus
113
114             return np.argmax(ucb_values)
115
116         def update(self, arm, reward):
117             self.counts[arm] += 1
118             self.total_count += 1
119             self.values[arm] += (reward - self.values[arm]) / self.
                counts[arm]
120

```

```

121
122 class ThompsonSamplingAgent:
123     def __init__(self, n_arms):
124         self.n_arms = n_arms
125         self.alpha = np.ones(n_arms)
126         self.beta = np.ones(n_arms)
127         self.counts = np.zeros(n_arms)
128         self.values = np.zeros(n_arms)
129
130     def select_action(self):
131         samples = np.random.beta(self.alpha, self.beta)
132         return np.argmax(samples)
133
134     def update(self, arm, reward):
135         self.counts[arm] += 1
136         self.values[arm] += (reward - self.values[arm]) / self.
            counts[arm]
137
138         normalized_reward = (reward + 3) / 6
139         normalized_reward = np.clip(normalized_reward, 0, 1)
140
141         self.alpha[arm] += normalized_reward
142         self.beta[arm] += (1 - normalized_reward)
143
144
145 def run_experiment(agent_class, agent_params, bandit, n_steps=1000,
    n_runs=100):
146     all_rewards = np.zeros((n_runs, n_steps))
147     all_arm_counts = np.zeros((n_runs, bandit.n_arms))
148
149     for run in range(n_runs):
150         agent = agent_class(**agent_params)
151         for step in range(n_steps):
152             arm = agent.select_action()
153             reward = bandit.pull(arm)
154             agent.update(arm, reward)
155             all_rewards[run, step] = reward
156
157         all_arm_counts[run] = agent.counts
158
159     avg_rewards = np.mean(all_rewards, axis=0)
160     cumulative_avg_rewards = np.cumsum(avg_rewards) / np.arange(1,
        n_steps + 1)
161     avg_arm_counts = np.mean(all_arm_counts, axis=0)

```

```

162
163     return cumulative_avg_rewards, avg_arm_counts
164
165
166 def task2_bandit():
167     print("任务2: 4臂赌博机实验")
168     print("【4臂赌博机设置】")
169     bandit = FourArmedBandit()
170     print(f"摇臂数量: {bandit.n_arms}")
171     print(f"各摇臂真实期望奖励: {bandit.true_means}")
172     print(f"奖励标准差: {bandit.std}")
173     print(f"最优摇臂: 摇臂{bandit.best_arm} (期望={bandit.true_means
        [bandit.best_arm]})")
174
175     n_steps = 1000
176     n_runs = 100
177
178     print(f"\n实验设置: 每轮{n_steps}步, 重复{n_runs}次取平均")
179
180     agents_config = {
181         '贪婪策略(epsilon=0.1)': (EpsilonGreedyAgent, {'n_arms': 4,
            'epsilon': 0.1}),
182         '贪婪策略(epsilon=0.01)': (EpsilonGreedyAgent, {'n_arms': 4,
            'epsilon': 0.01}),
183         '玻尔兹曼策略(T=0.5)': (BoltzmannAgent, {'n_arms': 4, '
            temperature': 0.5}),
184         'UCB策略(c=2)': (UCBAgent, {'n_arms': 4, 'c': 2.0}),
185         '汤普森采样': (ThompsonSamplingAgent, {'n_arms': 4}),
186     }
187
188     results = {}
189     for name, (agent_class, params) in agents_config.items():
190         print(f" 运行 {name}...")
191         cumulative_avg, arm_counts = run_experiment(agent_class,
            params, bandit, n_steps, n_runs)
192         results[name] = {
193             'cumulative_avg_rewards': cumulative_avg,
194             'arm_counts': arm_counts
195         }
196
197     fig, axes = plt.subplots(2, 2, figsize=(14, 10))
198
199     ax1 = axes[0, 0]
200     for name, data in results.items():

```

```

201         ax1.plot(data['cumulative_avg_rewards'], label=name,
202                 linewidth=1.5)
203     ax1.axhline(y=bandit.get_optimal_reward(), color='red',
204                 linestyle='--',
205                 label=f'最优期望奖励 ({bandit.get_optimal_reward()})', linewidth=2)
206     ax1.set_xlabel('训练步数', fontsize=12)
207     ax1.set_ylabel('平均累积奖励', fontsize=12)
208     ax1.set_title('各策略训练次数与平均累积奖励的关系', fontsize=14)
209     ax1.legend(loc='lower right', fontsize=9)
210     ax1.grid(True, alpha=0.3)
211
212     ax2 = axes[0, 1]
213     arm_names = [f'摇臂{i}\n(mu={bandit.true_means[i]})' for i in
214                 range(4)]
215     x = np.arange(4)
216     width = 0.15
217
218     for i, (name, data) in enumerate(results.items()):
219         bars = ax2.bar(x + i * width, data['arm_counts'], width,
220                       label=name)
221
222     ax2.set_xlabel('摇臂', fontsize=12)
223     ax2.set_ylabel('平均选中次数', fontsize=12)
224     ax2.set_title('各策略下每个摇臂的选中次数', fontsize=14)
225     ax2.set_xticks(x + width * 2)
226     ax2.set_xticklabels(arm_names)
227     ax2.legend(loc='upper left', fontsize=8)
228     ax2.grid(True, alpha=0.3, axis='y')
229
230     ax3 = axes[1, 0]
231     for name, data in results.items():
232         cumulative = data['cumulative_avg_rewards']
233         ax3.plot(cumulative, label=name, linewidth=1.5, alpha=0.7)
234     ax3.axhline(y=bandit.get_optimal_reward(), color='red',
235                 linestyle='--',
236                 label='最优期望奖励', linewidth=2)
237     ax3.set_xlabel('训练步数', fontsize=12)
238     ax3.set_ylabel('平均累积奖励', fontsize=12)
239     ax3.set_title('收敛过程对比', fontsize=14)
240     ax3.legend(loc='lower right', fontsize=9)
241     ax3.grid(True, alpha=0.3)
242     ax3.set_xlim(0, 200)

```

```

239 ax4 = axes[1, 1]
240 strategies = list(results.keys())
241 final_rewards = [results[s]['cumulative_avg_rewards'][-1] for s
    in strategies]
242 colors = plt.cm.Set3(np.linspace(0, 1, len(strategies)))
243
244 bars = ax4.barh(strategies, final_rewards, color=colors)
245 ax4.axvline(x=bandit.get_optimal_reward(), color='red',
    linestyle='--',
246             label=f'最优期望奖励', linewidth=2)
247 ax4.set_xlabel('最终平均累积奖励', fontsize=12)
248 ax4.set_title('各策略最终性能对比', fontsize=14)
249 ax4.legend(loc='lower right', fontsize=9)
250 ax4.grid(True, alpha=0.3, axis='x')
251
252 for bar, reward in zip(bars, final_rewards):
253     ax4.text(bar.get_width() + 0.02, bar.get_y() + bar.
        get_height()/2,
254             f'{reward:.3f}', va='center', fontsize=9)
255
256 plt.tight_layout()
257 plt.savefig('bandit_results.png', dpi=150, bbox_inches='tight')
258 plt.show()
259
260 print("【实验结果总结】")
261 print(f"\n各摇臂真实期望: {bandit.true_means}")
262 print(f"最优摇臂: 摇臂{bandit.best_arm} (期望={bandit.true_means
    [bandit.best_arm]})")
263 print("\n各策略最终平均累积奖励:")
264 for name, data in sorted(results.items(), key=lambda x: x[1]['
    cumulative_avg_rewards'][-1], reverse=True):
265     final_reward = data['cumulative_avg_rewards'][-1]
266     print(f"    {name}: {final_reward:.4f}")
267
268 print("\n各策略摇臂选中分布:")
269 for name, data in results.items():
270     counts = data['arm_counts']
271     total = sum(counts)
272     percentages = [c/total*100 for c in counts]
273     print(f"    {name}:")
274     for i, (cnt, pct) in enumerate(zip(counts, percentages)):
275         print(f"        摇臂{i}: {cnt:.1f}次 ({pct:.1f}%)")
276
277 print("\n图表已保存至: bandit_results.png")

```

```
278
279
280
281 if __name__ == "__main__":
282     task1_cartpole()
283     task2_bandit()
284
285     print("\n所有任务完成！")
```